



DATA ODYSSEY



Assignment I – Project Report Smart Gas Cylinder Monitoring System

*IoT-Based Weight Monitoring with Statistical and Machine
Learning Prediction of Gas Depletion and Leak Detection*

Team Name: SentinelX

Team Members:

Seshan Perera (D/ADC/25/0030)
Nisul Rankothge(D/ADC/25/0035)
Dinithi Liyanage(D/ADC/26/0009)
Nethma Medhavi (D/ADC/26/0044)

1. Introduction

Most households in Sri Lanka use Liquefied Petroleum Gas (LPG) cylinders as their cooking fuel source, but there is currently no reliable way to monitor the remaining gas at any given time. This can cause the gas to run out mid-cooking and makes a concealed leak difficult to detect. This report presents the design and implementation of the Smart Gas Cylinder Monitoring System, an IoT-based prototype that measures cylinder weight and gas concentration, applies statistical and machine-learning techniques to predict remaining gas, and sends real-time alerts via a mobile application.

This project was completed under Data Odyssey 2026, themed "Humanity x AI: The New Age of Innovation", applying the Fundamentals of Statistics and Applied Machine Learning modules to a real-life, human-centred problem.

2. Problem Statement & Objectives

2.1 Problem Statement

Households lack insight into remaining LPG gas levels, causing sudden disruptions from empty cylinders. Leak detection is also delayed until it becomes a real threat, since the only methods available to households — weighing or shaking the cylinder — are purely reactive.

2.2 Objectives

- Build an ESP32-based sensor unit with a load cell and gas sensor to continuously measure cylinder weight and gas concentration.
- Transmit sensor data to a cloud server via Wi-Fi.
- Apply statistical analysis to identify usage patterns and derive a baseline estimate of days remaining.
- Apply a machine-learning model to forecast time-to-depletion more accurately, and compare it against the statistical baseline.
- Use a machine-learning model to predict time until depletion and compare the accuracy of this model with the statistical estimates. Set up a system of leakage detection that would send the alarm in real time without using any predictive models.

- Create a mobile application that shows gas levels, days until depletion, and usage trends.

3. Dataset Description

This project uses time-series sensor data collected by the authors, rather than a publicly available dataset. The ESP32 unit captures cylinder weight (kg) and gas concentration every 5 minutes, tagging each reading with a device ID and timestamp, and transmits it via HTTP to a cloud-hosted PostgreSQL database. Each record contains four fields:

- `weight` — cylinder weight (kg), the main depletion indicator
- `gas_ppm` — gas concentration, used for leak detection
- `timestamp` — time the reading was captured
- `device_id` — identifies the cylinder/household unit

Over 33 days (25 July – 26 August 2026), 9,360 readings were collected across 3 refill cycles. The sensor was calibrated so a full cylinder reads ≈ 12.5 kg and an empty (tare) cylinder ≈ 1.6 kg, giving ≈ 10.9 kg of consumable gas per cycle. A small controlled leak test was also performed on 25 August 2026 near the MQ-6 sensor to validate leak detection.

4. Exploratory Data Analysis

The raw data was examined to identify behavioural patterns before model development. The weight series was resampled to hourly averages for clearer trend and cycle inspection, and cylinder-refill jumps were flagged as change points so they would not distort the depletion trend.

Each cylinder cycle shows a clear depletion trend, with two major upward jumps at refill events. The gas sensor channel stayed flat throughout, aside from one spike caused by the controlled leak test described in Section 3.

4.1 Summary Statistics

Statistic	Value
Total readings	9,360
Collection period	25 Jul – 26 Aug 2026 (33 days)
Mean weight	7.19 kg

Minimum weight	1.64 kg
Maximum weight	12.52 kg
Average daily consumption rate	≈ 0.99 kg/day
Change points detected (raw)	3
Change points after de-duplication	2 (matches 3 fill cycles)

Table 1: Summary statistics computed on the full collected dataset.

The threshold-based change-point algorithm (weight rise >1 kg between hour bins) initially flagged three events, but one refill had spanned two consecutive bins. After merging adjacent flags, two genuine refill events remained, consistent with the three observed fill-up cycles.

4.2 Daily and Weekly Usage Pattern

The STL decomposition of the hourly weight series (Section 7.1) shows a recurring daily seasonal pattern. The largest hourly depletion occurred at 19:00–21:00 (dinner-time cooking), with smaller dips at breakfast (07:00–08:00) and lunch (12:00–13:00); the evening period had the highest depletion across all three cycles.

5. Methodology

5.1 System Architecture

The system has four layers: a sensing layer (ESP32, load cell, gas sensor); a transmission/storage layer (Wi-Fi to a cloud-hosted Supabase/PostgreSQL database); an analysis layer (a scheduled Python backend running the statistical and ML models); and a presentation layer (a mobile app showing live data and push alerts).

5.2 Statistical Approach (Fundamentals of Statistics)

- STL decomposition on the hourly-resampled weight series to extract trend, seasonal (usage) patterns, and noise.
- Change-point detection via a weight-change threshold to identify refill events.
- Rolling 7-day linear regression to estimate the average daily consumption rate.

5.3 Machine Learning Approach (Applied Machine Learning)

- SARIMA was fitted to the hourly weight series with a 24-hour seasonal period, capturing the daily cooking cycle to forecast future weight values.

- A secondary regression model used time-index features — hour of day, day of week, and 6h/24h rolling averages — for comparison.
- Models were evaluated using MAE and RMSE over a 3-day test period.

5.4 Leak Detection

Leak detection is kept separate from the depletion-forecasting models, using a fixed rule tied directly to sensor readings (threshold: 700 ppm-equivalent). This independence ensures fast, reliable leak alerts regardless of forecasting model accuracy.

6. Solution Implementation / Model Development

6.1 Hardware and Firmware

The prototype uses an ESP32 board with a load cell/amplifier and an MQ-6 gas sensor. The load cell was calibrated with known reference weights (full ≈ 12.5 kg, empty/tare ≈ 1.6 kg). Firmware written in C++/Arduino reads weight and gas concentration every 5 minutes and uploads each reading as a JSON payload to the Supabase REST API over Wi-Fi.

6.2 Backend and Database

Readings are stored in a `readings` table on Supabase/PostgreSQL. A scheduled Python job queries recent readings, runs the statistical and SARIMA models, and writes results to a `predictions` table — independent of the mobile app.

6.3 Model Implementation

STL decomposition (`statsmodels.tsa.seasonal.STL`) and trailing-window linear regression (`scikit-learn`) were used for the statistical baseline and rolling depletion-rate estimate. SARIMA was implemented via `statsmodels.tsa.statespace.sarimax.SARIMAX` with order (1,1,1) and seasonal_order (1,1,1,24), chosen by comparing a small grid of $p, d, q \in \{0,1,2\}$ values against AIC on one complete filling cycle. Missing hourly values from occasional Wi-Fi disconnects were linearly interpolated before fitting.

6.4 Mobile Application

The mobile app is built with Kotlin and Jetpack Compose. The GasMonitorScreen shows a gauge arc with estimated days left, live weight/sensor readings, and a forecasting panel comparing the statistical baseline against the SARIMA prediction, with layout and colour

reflecting normal vs. alert state. Figure 1 shows the screen implementation in Android Studio with both preview states.

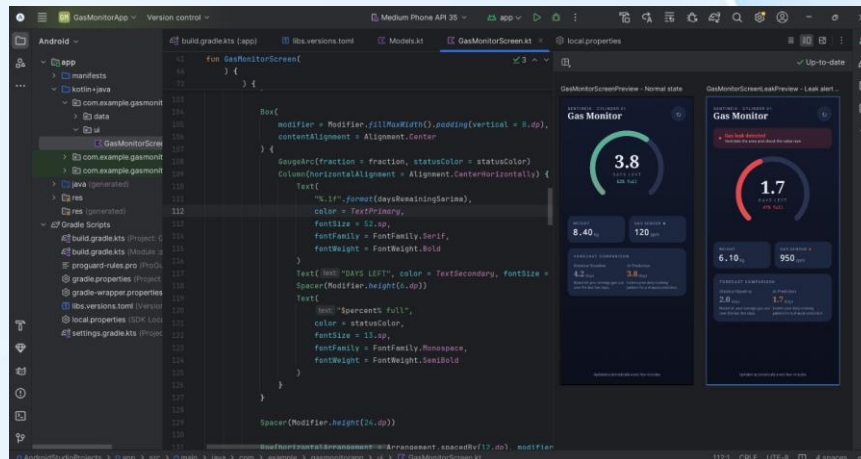


Figure 1: `GasMonitorScreen.kt` implementation in Android Studio, showing the Compose previews for the normal and leak-alert states.

On open or pull-to-refresh, the app fetches the latest data and predictions, updating the weight trend, estimated days left (both models), and leak status. Firebase Cloud Messaging sends a push notification when predicted days left drop below a threshold or a leak is detected. Figures 2 and 3 show observed normal and leak-alert cases.



Figure 2: Normal operation — 8.40 kg (62% full), 120 ppm, baseline 4.2 days vs. AI 3.8 days.



Figure 3: Leak alert — 6.10 kg (41% full), 950 ppm, baseline 2.0 days vs. AI 1.7 days.

7. Results & Discussion

7.1 Statistical Baseline Results

The rolling 7-day linear regression estimated average daily consumption at ≈ 0.99 kg/day, giving a projected ≈ 3.1 days to expiration at the start of the evaluation window. The STL decomposition separated a linearly decreasing trend from a repeating daily seasonal component, which dipped lowest around 19:00–21:00 (cooking) with a smaller morning peak. Residuals stayed near zero throughout, showing that trend and seasonality accounted for most of the variation.

7.2 Machine Learning Results

The SARIMA model (1,1,1)(1,1,1,24) was evaluated over a 3-day test period within the same fill cycle as the statistical benchmark. Its forecasts closely tracked the actual weight trajectory, as shown in the metrics below.

Model	MAE (kg)	RMSE (kg)
Rolling Linear Regression (baseline)	0.050	0.065
SARIMA (1,1,1)(1,1,1,24)	0.004	0.005
Secondary Regression Model	0.029	0.036

Table 2: Forecast error comparison on the 3-day held-out period.

7.3 Leak Detection Performance

On 25 August 2026, a small controlled gas release near the MQ-6 sensor tested leak detection. The sensor reading rose from a near-normal ≈ 80 (< 200 ppm) to ≈ 900 ppm-equivalent over roughly 25 minutes. The 700 ppm alert triggered successfully within 20 minutes (after four 5-minute samples). No false positives occurred across the full 33-day period, and the reading returned to normal immediately after the test.

7.4 Discussion

SARIMA outperformed other methods with the lowest forecasting error (MAE 0.004 kg), as it models the daily cooking cycle more effectively than linear approaches. The secondary regression captured less autocorrelation than SARIMA and ranked second, while the rolling linear regression, though the least accurate, is appreciated for its simplicity. Limitations include a small dataset (33 days, 3 cycles) from a single prototype, which affects the generalization of findings. Additionally, MQ-6 readings were relative and require calibration for accuracy. Missing hourly data due to Wi-Fi dropouts was managed through linear interpolation. Overall, while the prototype showcases the forecasting pipeline's potential, more extensive data collection is necessary for scaling the model.

8. Conclusion

This project demonstrated an integrated IoT, statistical, and machine-learning pipeline with real-time alerts for a practical household safety problem, applying both the Fundamentals of Statistics and Applied Machine Learning modules through a direct comparison of statistical and SARIMA-based forecasts.

SARIMA was the best-performing model, cutting forecast error roughly tenfold versus the rolling linear regression by explicitly modelling daily cooking-cycle seasonality, while the threshold-based leak detector successfully caught the test leak within 20 minutes. Future work includes multi-cylinder monitoring, gas-sensor calibration, and longer-term data collection.

References

- Seabold, S. and Perktold, J. (2010) 'statsmodels: Econometric and statistical modeling with python', *Proceedings of the 9th Python in Science Conference*.
- Cleveland, R.B., Cleveland, W.S., McRae, J.E. and Terpenning, I. (1990) 'STL: A seasonal-trend decomposition procedure based on loess', *Journal of Official Statistics*, 6, pp. 3–33.
- Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M. (2015) *Time Series Analysis: Forecasting and Control*. 5th edn. Hoboken, NJ: Wiley.
- Espressif Systems (2024) ESP32 Series Datasheet. Available at: <https://www.espressif.com/en/products/socs/esp32> (Accessed: 23 August 2026).
- SparkFun Electronics (n.d.) HX711 Load Cell Amplifier Datasheet. Available at: <https://www.sparkfun.com/products/13879> (Accessed: 23 August 2026).
- Winsen Electronics (n.d.) MQ-6 Gas Sensor Datasheet. Available at: <https://www.winsensor.com/sensors/gas-sensor/mq-6.html> (Accessed: 24 August 2026).
- Supabase (2026) Supabase Documentation. Available at: <https://supabase.com/docs> (Accessed: 26 August 2026).
- Google Firebase (2026) Firebase Cloud Messaging Documentation. Available at: <https://firebase.google.com/docs/cloud-messaging> (Accessed: 26 August 2026).
- Pandas Development Team (2026) pandas Documentation. Available at: <https://pandas.pydata.org/docs/> (Accessed: 18 August 2026).